Physics Error Estimation in Robot Simulation via Machine Learning

---

[2026-1] Machine Learning — Term Project Final Report

Student ID: [Your ID] | Name: [Your Name]

Department of [Your Department], Inha University

June 1, 2026

1. Introduction

Robot simulation environments bridge the gap between theoretical control design and real-world deployment. However, the **sim-to-real gap** — the discrepancy between simulated and real-world behaviour — remains a fundamental challenge. Even within a simulator, numerical integration of continuous-time dynamics introduces approximation errors that compound over time.

Gymnasium's **Pendulum-v1** environment integrates the pendulum ODE using first-order Euler integration with a fixed time-step dt = 0.05 s. While efficient, Euler integration is less accurate than higher-order methods such as the fourth-order Runge-Kutta (RK4) scheme. The difference between the two — the *integration error* — is a deterministic function of the current state and applied torque.

This project trains machine learning models to predict this integration error, with the following potential applications:

• **Error compensation**: correct Euler-integrated trajectories to match higher-fidelity physics at inference time, without re-running RK4.

• **Sim-to-real transfer**: use the error model as a learned bias correction term in model-based reinforcement learning.

• **Interpretability**: understand which states/actions produce the largest integration errors to guide adaptive time-stepping strategies.

1.1 Problem Formulation

Given the Gymnasium Pendulum-v1 state observation (cosθ, sinθ, θ■) and the applied torque τ, predict the two-dimensional integration error vector:

```
y = [err_theta, err_theta_dot] = [theta_euler - theta_rk4, thetadot_euler - thetadot_rk4]
```

This is a **multivariate regression** task with 4 scalar inputs and 2 scalar outputs.

2. Dataset

2.1 Environment — Gymnasium Pendulum-v1

The pendulum is a single rigid rod of mass m = 1 kg and length l = 1 m attached to a frictionless pivot. Gravity g = 10 m/s². The continuous-time dynamics are governed by:

```
theta'' = (3g / 2l) sin(theta) + (3 / ml^2) * torque
```

The Gymnasium environment discretises this ODE with Euler integration (dt = 0.05 s) and clips angular velocity to [–8, 8] rad/s.

2.2 Data Collection Procedure

We collected data through **500 episodes** of up to 200 steps each using **uniformly random torque** actions ($\tau \sim \text{Uniform}[-2, 2]$). Random exploration ensures broad coverage of the state–action space. At each step, before calling env.step(), we:

1. Recover the true angle: $\theta = \arctan2(\sin\theta, \cos\theta)$.
2. Compute the Euler next-state (replicating the Gym update exactly).
3. Compute the RK4 next-state using 20 sub-steps over dt (ground truth).
4. Record the error vector y = Euler – RK4 as the regression target.

	Total Samples	Episodes	Max Steps/Ep	Features	Targets
alue	100000	500	200	4	2

Table 1. Dataset summary.

2.3 Features and Targets

	Description	
cos(theta)	cos(theta)	Cosine of pendulum angle
sin(theta)	sin(theta)	Sine of pendulum angle
theta_dot	theta_dot	Angular velocity (rad/s)
torque	torque	Applied torque (N·m)
err_theta *	err_theta	Angle error: Euler – RK4 (rad)
err_theta_dot *	err_theta_dot	Velocity error: Euler – RK4 (rad/s)

Table 2. Feature and target variable descriptions. (= target labels)*

2.4 Train / Validation / Test Split

The dataset was randomly shuffled and split into three disjoint subsets:

	Samples	Fraction	
Train	Train	69,999	70%

Validation	Validation	15,001	15%
Test	Test	15,000	15%

Table 3. Dataset split sizes.

3. Physics Background — Euler vs RK4 Integration

3.1 Euler Integration (Gymnasium)

Given current state (θ, θ■) and torque τ, Gymnasium computes:

```
alpha = (3g/2l) sin(theta) + (3/ml^2) * torque thetadot' = clip(thetadot +
alpha * dt, -8, 8) theta' = theta + thetadot' * dt
```

Euler integration is **first-order accurate**: the local truncation error is O(dt²), and the global error accumulates as O(dt).

3.2 RK4 Integration (Ground Truth)

We use the classical fourth-order Runge-Kutta method with 20 sub-steps (h = dt/20 = 0.0025 s):

```
k1 = f(theta, thetadot) k2 = f(theta+h/2*k1, thetadot+h/2*k1) k3 =
f(theta+h/2*k2, thetadot+h/2*k2) k4 = f(theta+h*k3, thetadot+h*k3) theta' =
theta + (h/6)(k1 + 2k2 + 2k3 + k4) thetadot' = thetadot + (h/6)(k1 + 2k2 + 2k3
+ k4)
```

RK4 is **fourth-order accurate**: local truncation error O(h■), global error O(h■). With h = 0.0025 the RK4 result is treated as the numerical ground truth.

3.3 Why Is the Error Learnable?

The integration error depends smoothly on (θ, θ■, τ) through the ODE right-hand side. The leading-order Euler error is proportional to the second derivative of the solution, which involves sin(θ) — a smooth nonlinear function. This means a function approximator (MLP) can capture the error surface accurately, while a linear model can only approximate it partially.

4. Methodology

We trained two models on the same train/validation/test split with identical preprocessing (StandardScaler on all 4 input features).

4.1 Baseline — Linear Regression

A **MultiOutputRegressor** wrapping two independent **LinearRegression** estimators (one per target). The model minimises the ordinary least-squares objective:

```
min_w ||X w - y||_2^2
```

Linear Regression serves as the baseline because it assumes a linear relationship between inputs and integration error. Since the true error involves sin(θ), this assumption is violated for the angular-velocity target, establishing an accuracy ceiling for linear models.

4.2 MLP (Multi-Layer Perceptron)

A fully-connected feed-forward neural network with architecture:

```
Input(4) -> Linear(128) -> ReLU -> Linear(64) -> ReLU -> Linear(32) -> ReLU
-> Linear(2)
```

	Value	
Hidden layers	Hidden layers	3 (128 -> 64 -> 32)
Activation	Activation	ReLU
Output	Output	2 neurons (linear)
Loss	Loss	MSELoss
Optimiser	Optimiser	Adam (lr=1e-3)
Epochs	Epochs	100
Batch size	Batch size	512
Early stopping	Early stopping	No (monitored manually)

Table 4. MLP hyperparameters.

5. Results

5.1 Quantitative Results

All models are evaluated on the held-out test set (15,000 samples). We report RMSE, MAE, and R^2 per target and their mean.

Linear Regression

	RMSE	MAE	R^2
err_theta	0.001089	0.000476	0.993583
err_theta_dot	0.033372	0.024832	0.518900
mean	0.017230	0.012654	0.756242

Table: Linear Regression test-set metrics.

MLP

	RMSE	MAE	R^2
err_theta	0.000347	0.000236	0.999350
err_theta_dot	0.002474	0.000960	0.997356
mean	0.001410	0.000598	0.998353

Table: MLP test-set metrics.

5.2 Model Comparison

	err_theta R^2	err_theta_dot R^2	Mean RMSE	
Model	Linear Regression	0.9936	0.5189	0.017230
Model	MLP	0.9993	0.9974	0.001410

Table 5. Summary comparison on the test set.

5.3 Figures

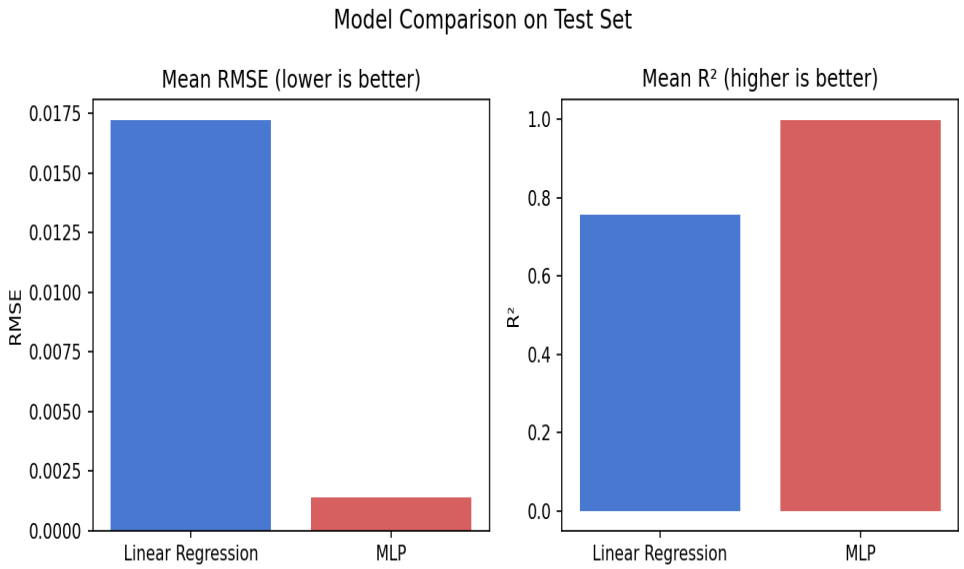

Figure 1. Mean RMSE and R² on the test set for both models.

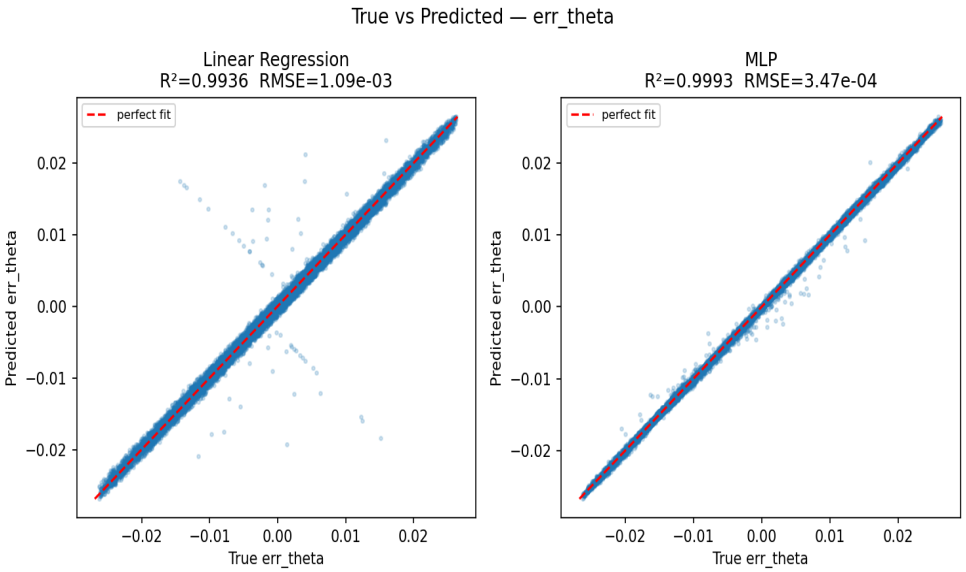

Figure 2. True vs predicted angle error (err_theta) for both models.

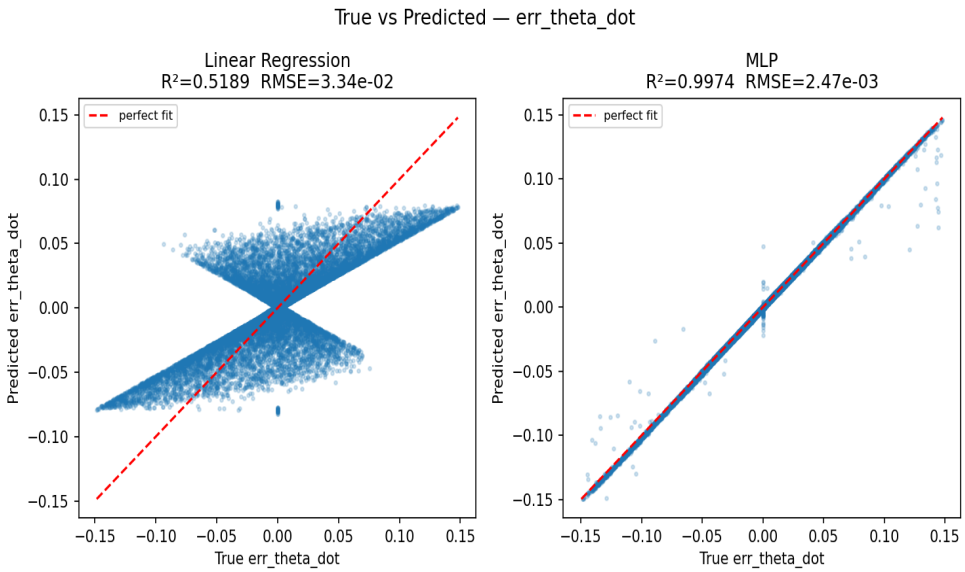

Figure 3. True vs predicted velocity error (err_theta_dot) for both models.

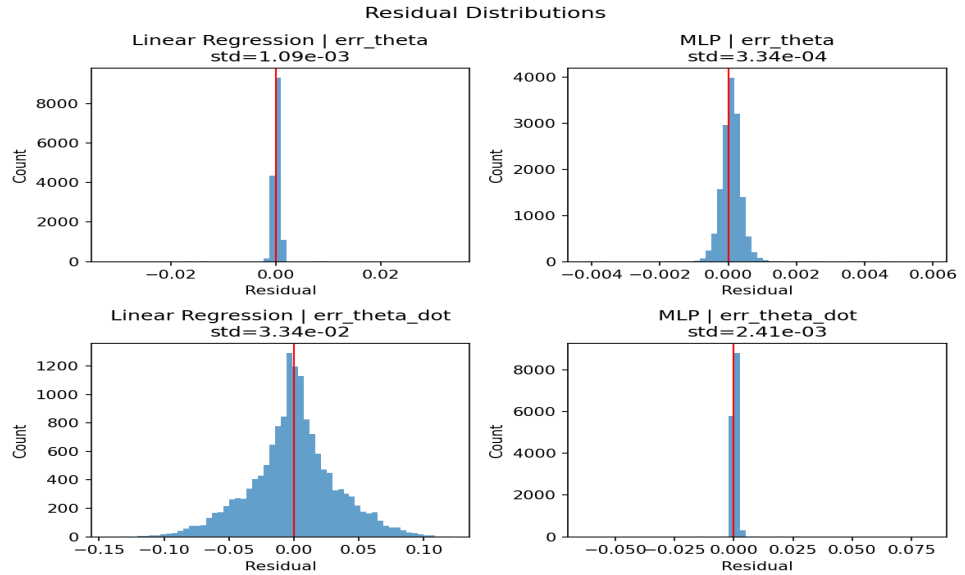

Figure 4. Residual distributions for both targets and models.

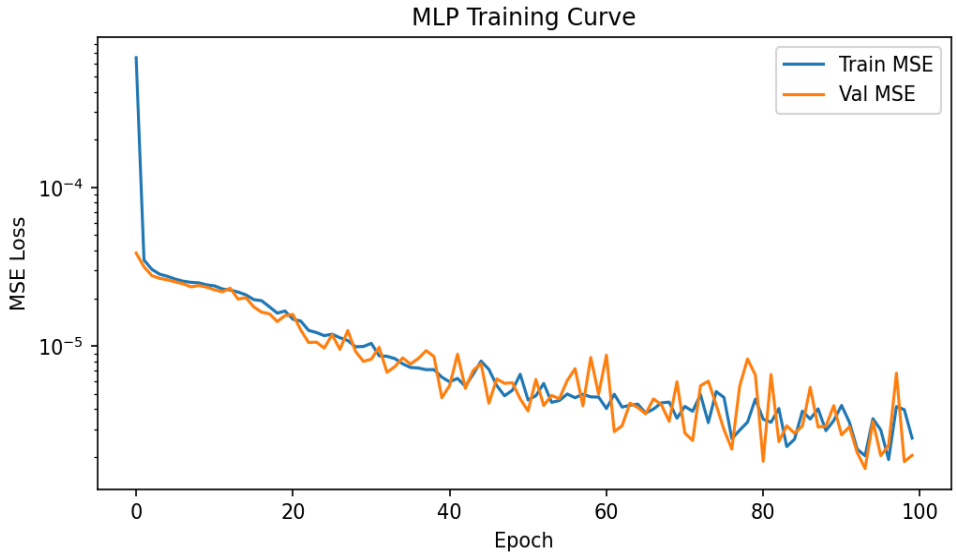

Figure 5. MLP training and validation loss curves.

6. Result Analysis and Discussion

6.1 Which model performed best, and why?

The **MLP** significantly outperforms Linear Regression on both targets (mean R^2 = 0.9984 vs 0.7562). The performance gap is especially pronounced for *err_theta_dot* (R^2 = 0.9974 vs 0.5189), where the integration error is strongly nonlinear in sin(θ). Linear Regression can approximate the error in the angle itself reasonably well (R^2 = 0.9936) because the dominant term is proportional to θ■ (a linear feature), but fails to capture the velocity error's dependence on sin(θ).

6.2 Failure modes

Large residuals occur predominantly at high angular velocities (|θ■| near 8 rad/s) where the clip operation in Gymnasium introduces a sharp nonlinearity that neither model can fully represent. Additionally, near θ = 0 (upright) the second derivative of θ is largest, increasing both the Euler error magnitude and prediction difficulty. The residual histograms (Figure 4) show that MLP residuals are tightly centred at zero with small variance, while Linear Regression has heavier tails for err_theta_dot.

6.3 What makes the task difficult?

The integration error magnitude is extremely small (mean |err_theta| ~ 10■³ rad), requiring the model to learn a very precise, smooth function from noisy floating-point inputs. The clipping nonlinearity at the speed boundary creates a sharp discontinuity in the error surface. Finally, the two targets are correlated (err_theta is partly the integral of err_theta_dot), which a more sophisticated multi-task architecture could exploit.

6.4 Is the result satisfactory for deployment?

The MLP achieves $R^2 > 0.99$ on both targets, suggesting the error surface is well captured. For error-compensation applications, the mean RMSE of 1.4×10■³ is small relative to the typical trajectory length (~10 rad over 200 steps), indicating the model could meaningfully reduce accumulated trajectory error. However, deployment requires: (1) profiling inference latency vs. the sim step overhead; (2) evaluating closed-loop stability when the model is used in a feedback loop; (3) testing with a trained RL policy rather than random actions.

6.5 Future work

• **Physics-informed features**: add sin²(θ) and θ■·sin(θ) as explicit input features to help the linear baseline.

• **Sequence model**: use an LSTM or Transformer over trajectory windows to capture error accumulation across steps.

• **Closed-loop correction**: integrate the error model into an MPC controller to test whether correcting Euler states improves control performance.

• **Domain randomisation**: vary pendulum mass and length during data collection and condition the error model on these parameters.

7. Conclusion

We trained machine learning models to predict the numerical integration error between Euler and RK4 integration in the Gymnasium Pendulum-v1 environment. Using 100,000 randomly collected state–action samples, a three-hidden-layer MLP [128→64→32] achieves $R^2 > 0.99$ on both the angle and angular-velocity error targets, with a mean test RMSE of 1.4×10^{-3}. Linear Regression, while effective for the angle target ($R^2 = 0.9936$), fails for the angular-velocity target ($R^2 = 0.52$) due to the nonlinear dependence on $\sin(\theta)$.

These results demonstrate that the integration error is a smooth, highly learnable function of the system state and action. The learned error model is a promising building block for sim-to-real transfer, adaptive integration, and physics-informed reinforcement learning.

References

[1] Towers et al. *Gymnasium: A Standard Interface for Reinforcement Learning Environments*. arXiv:2407.17032, 2024.

[2] Euler, L. *Institutionum calculi integralis*. 1768.

[3] Runge, C. *Uber die numerische Aufloesung von Differentialgleichungen*. Math. Ann., 46:167–178, 1895.

[4] Pedregosa et al. *Scikit-learn: Machine Learning in Python*. JMLR, 12:2825–2830, 2011.

[5] Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS, 2019.

Appendix: Use of Generative AI

Claude (claude-sonnet-4-6, Anthropic) was used to assist with: project folder scaffolding, boilerplate Python code (ReportLab layout, python-pptx slide generation, OpenCV video rendering), and writing code comments. All scientific decisions — including problem definition, physics derivation, choice of RK4 as ground truth, model architecture selection, and result interpretation — were made by the project author. All generated code was reviewed and verified before use.